GUÍA GENERAL Y EJERCICIOS DEL EXAMEN PARCIAL

Asignatura: Estructuras de Datos (Curso 2026-2027) | Planificación Oficial (42 Estudiantes)

1. Indicaciones Generales de la Entrega

Cada equipo deberá entregar un enlace público a un repositorio de GitHub que contenga los siguientes elementos obligatorios:

- **Proyecto Java Funcional:** Código fuente limpio con solución orientada a objetos en Java. Debe incluir casos de prueba tanto **positivos** (flujos normales) como **negativos** (listas vacías, elementos no encontrados, manejo de excepciones).
- **Presentación para Exposición Oral:** Archivo en formato PDF o diapositivas (PowerPoint, Google Slides o Canva) integrado en el repositorio o enlazado claramente en el README.
- **Archivo README.md Documentado:** Explicación clara en lenguaje natural de la problemática, la estructura de datos empleada, el detalle de casos de prueba ejecutados y una **Declaración explícita sobre el uso de Inteligencia Artificial** (detallando en qué partes específicas se empleó, ej. pruebas, refactorización, redacción).

2. Cronograma y Planificación de Exposiciones

Las exposiciones cuentan con 42 estudiantes distribuidos en 9 equipos (6 equipos de 5 integrantes y 3 equipos de 4 integrantes), organizados en 2 turnos de clase (180 minutos totales):

Turno	Horario	Equipo	Integrantes
1.º Turno	1:00 PM – 1:20 PM	**Equipo 1 (5)**	Yaisel Pinos Durán, Merlyn Sosa Borges, Geonel Sánchez, Jorge Luis Reyes Pérez, Ray Sanjudo Mesa
1.º Turno	1:20 PM – 1:40 PM	**Equipo 2 (5)**	Katherine Alvarez Amador, Adrian Jesus Pelaez Rodriguez, Adrian Enrique Reinoso Gómez, Edgar Guillermo Hernández Domínguez, Alejandro Javier Montesino Ruiz
1.º Turno	1:40 PM – 2:00 PM	**Equipo 3 (5)**	Ariela Ivón Corrales Rodríguez, Mélany Fernández Díaz, Miriam Elena Pérez Hdez, Julio César León Gato, Ronal Hernández Castillo
1.º Turno	2:00 PM – 2:20 PM	**Equipo 4 (5)**	Emily Elena Concha Morejón, Martha Alina Montané Arias, César Paredes Real, Dario Riverol Herrera, Lazaro Gabriel Garcia Enriquez
Ajuste	2:20 PM – 2:30 PM	**Receso**	Margen de ajuste e inicio del segundo turno
2.º Turno	2:30 PM – 2:48 PM	**Equipo 5 (5)**	Lauren Rosa Barrios Bosque, Liz Anel López Vives, Osmany Manuel Ungo Seara, Jorge Luis Hernández Avila, José Javier Fernández Bravo
2.º Turno	2:48 PM – 3:06 PM	**Equipo 6 (5)**	Pedro Antonio Palacios González, Jesus Daniel Bruguera Gallardo, Leonardo Roque Moreno, Caleb Blanco Morejón, Christian M. Febles Suárez
2.º Turno	3:06 PM – 3:24 PM	**Equipo 7 (4)**	Onel Alejandro Olivera Díaz, Pedro Luis Sarmiento Torres, Daniela González Alemán, María Fernanda Rodríguez Díaz
2.º Turno	3:24 PM – 3:42 PM	**Equipo 8 (4)**	Anyelo Monterrey, Carlos Daniel Hernández Díaz, Laura García Miguel, Alejandro Marrinez Gonzalez
2.º Turno	3:42 PM – 4:00 PM	**Equipo 9 (4)**	Isnaelys Temerez Rodriguez, Dalís Aladro Acosta, Adrián Felipe Armas García, Carlos Daniel Borrego Chang

3. Enunciados de Ejercicios por Equipo

Cada integrante del equipo responderá e implementará un método/funcionalidad específica asignada.

EQUIPO 1 (1:00 PM – 1:20 PM) — Estructura: *Lista Simplemente Enlazada*

Integrantes: Yaisel Pinos Durán, Merlyn Sosa Borges, Geonel Sánchez, Jorge Luis Reyes Pérez, Ray Sanjudo Mesa

Problemática: Farmacia 'FarmaVida'

En la farmacia 'FarmaVida' se desea controlar los datos de los medicamentos disponibles. De cada medicamento se conoce: código, nombre, categoría (Analgésico, Antibiótico, Antiinflamatorio, Vitamina), stock y precio.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **MedicamentosAgotados():** Devuelve los nombres de los medicamentos cuyo stock sea menor o igual a 5 unidades.

b) **PromedioPrecioPorCategoria(String categoria):** Devuelve el precio promedio de los medicamentos de la categoría especificada (0.0 si no existen).

c) **ActualizarStock(String codigo, int cantidadAgregada):** Busca un medicamento por su código y le suma la cantidad entregada. Devuelve true si existe y false si no.

d) **ListarPreciosInversos():** Devuelve la lista de precios en orden inverso al que fueron ingresados.

e) **AplicarAumentoGeneral(double porcentaje):** Incrementa el precio de todos los medicamentos en el porcentaje dado y devuelve la cantidad de medicamentos modificados.

Casos de Prueba Sugeridos: Insertar: (M01, Paracetamol, Analgésico, 15, 2.50), (M02, Amoxicilina, Antibiótico, 3, 12.00), (M03, Ibuprofeno, Antiinflamatorio, 2, 4.50), (M04, Vitamina C, Vitamina, 20, 5.00), (M05, Azitromicina, Antibiótico, 5, 15.00).

EQUIPO 2 (1:20 PM – 1:40 PM) — Estructura: *Lista Simplemente Enlazada*

Integrantes: Katherine Alvarez Amador, Adrian Jesus Pelaez Rodriguez, Adrian Enrique Reinoso Gómez, Edgar Guillermo Hernández Domínguez, Alejandro Javier Montesino Ruiz

Problemática: Aerolínea 'SkyFly'

En la aerolínea 'SkyFly' se desea controlar los datos de los vuelos programados. De cada vuelo se conoce: númeroVuelo, destino, estado (A tiempo, Demorado, Cancelado) y duraciónMinutos.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **VuelosDemorados():** Devuelve un listado con los números de vuelo de los vuelos con estado 'Demorado'.

b) **MostrarOrdenadosPorDuracion():** Devuelve la información de los vuelos ordenados por duración de menor a mayor.

c) **CancelarVuelosADestino(String destino):** Elimina todos los vuelos hacia el destino especificado y devuelve la cantidad de vuelos eliminados.

d) **CambiarEstadoVuelo(String numeroVuelo, String nuevoEstado):** Busca un vuelo por su número y actualiza su estado. Devuelve true si se encontró o false en caso contrario.

e) **BuscarVueloMasLargo():** Devuelve la información del vuelo con mayor duración en minutos registrado en la lista.

Casos de Prueba Sugeridos: Insertar: (SK101, Madrid, A tiempo, 120), (SK102, Cancún, Demorado, 210), (SK103, Madrid, Cancelado, 120), (SK104, Miami, Demorado, 90), (SK105, Bogotá, A tiempo, 150).

EQUIPO 3 (1:40 PM – 2:00 PM) — Estructura: *Lista Simplemente Enlazada*

Integrantes: Ariela Ivón Corrales Rodríguez, Mélany Fernández Díaz, Miriam Elena Pérez Hdez, Julio César León Gato, Ronal Hernández Castillo

Problemática: Centro Deportivo 'FitGym'

En el centro deportivo 'FitGym' se desea gestionar el registro de socios. De cada socio se conoce: idSocio, nombre, tipoMembresía (Básica, Premium, VIP) y díasRestantes.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **MembresiasPorVencer(int diasLimite):** Devuelve las IDs de los socios cuyos días restantes sean menores o iguales a diasLimite.

b) **MostrarOrdenadosPorDias():** Devuelve la información de los socios ordenados de mayor a menor según los días restantes.

c) **CantidadSociosPorMembresia():** Devuelve un String con el conteo de socios agrupados por tipo de membresía.

d) **RenovarMembresia(String idSocio, int diasAdicionales):** Busca un socio por su ID y le suma los días adicionales. Devuelve true si existe y false en caso contrario.

e) **EliminarSociosSinDias():** Elimina de la lista a todos los socios cuyos días restantes sean iguales a 0 y retorna la cantidad de registros eliminados.

Casos de Prueba Sugeridos: Insertar: (S01, Carlos, Básica, 5), (S02, Ana, Premium, 30), (S03, Pedro, VIP, 2), (S04, María, Básica, 15), (S05, Lucía, Premium, 60).

EQUIPO 4 (2:00 PM – 2:20 PM) — Estructura: *ArrayList*

Integrantes: Emily Elena Concha Morejón, Martha Alina Montané Arias, César Paredes Real, Dario Riverol Herrera, Lazaro Gabriel Garcia Enriquez

Problemática: Biblioteca Universitaria 'BiblioTech'

En la biblioteca 'BiblioTech' se necesita un módulo para gestionar los préstamos de libros. De cada libro prestado se registra: código, título, categoría (Ingeniería, Historia, Literatura, Ciencias) y díasDePréstamo.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **LibrosVencidos(int diasLimite):** Devuelve un listado de títulos cuya cantidad de días de préstamo supere el límite indicado.

b) **PromedioDiasPorCategoria(String categoria):** Calcula y devuelve el promedio de días de préstamo para los libros de una categoría determinada.

c) **RenovarPrestamo(String codigo, int diasAdicionales):** Busca un libro por su código y le suma los días adicionales. Retorna true si se encontró o false en caso contrario.

d) **EliminarLibrosPorCategoria(String categoria):** Elimina del catálogo todos los libros pertenecientes a la categoría especificada y retorna la cantidad eliminada.

e) **BuscarLibroConMasDias():** Devuelve el título y los días del libro que tiene asignado el mayor tiempo de préstamo.

Casos de Prueba Sugeridos: Insertar: (L01, Java a Fondo, Ingeniería, 15), (L02, Cien Años de Soledad, Literatura, 8), (L03, Estructuras de Datos, Ingeniería, 22), (L04, Breve Historia Universal, Historia, 30).

EQUIPO 5 (2:30 PM – 2:48 PM) — Estructura: *ArrayList*

Integrantes: Lauren Rosa Barrios Bosque, Liz Anel López Vives, Osmany Manuel Ungo Seara, Jorge Luis Hernández Avila, José Javier Fernández Bravo

Problemática: Tienda en Línea 'TechMarket'

El comercio electrónico 'TechMarket' requiere administrar su catálogo de productos informáticos. De cada producto se guarda: id, nombre, marca y precio.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **FiltrarPorRangoPrecio(double precioMin, double precioMax):** Retorna los nombres de los productos cuyo precio esté en el rango inclusivo.

b) **AplicarDescuentoMarca(String marca, double porcentaje):** Reduce el precio en el porcentaje dado a todos los productos de la marca indicada.

c) **BuscarProductoBarato():** Devuelve la información del producto con el precio más bajo registrado.

d) **ReponerStockProducto(String id, double nuevoPrecio):** Actualiza el precio de un producto dado su ID. Retorna true si existía o false en caso contrario.

e) **PromedioPrecioPorMarca(String marca):** Devuelve el precio promedio de todos los productos de la marca indicada (0.0 si no existen productos).

Casos de Prueba Sugeridos: Insertar: (P01, Laptop Gaming, ASUS, 1200.0), (P02, Mouse Inalámbrico, Logitech, 25.0), (P03, Teclado Mecánico, ASUS, 80.0), (P04, Monitor 24", Samsung, 180.0).

EQUIPO 6 (2:48 PM – 3:06 PM) — Estructura: *ArrayList*

Integrantes: Pedro Antonio Palacios González, Jesus Daniel Bruguera Gallardo, Leonardo Roque Moreno, Caleb Blanco Morejón, Christian M. Febles Suárez

Problemática: Gestión Académica 'EduControl'

El departamento de informática necesita procesar las notas del parcial de sus asignaturas. De cada estudiante inscrito se registra: idEstudiante, nombre, notaParcial (0 a 100) y asistencia (porcentaje).

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **EstudiantesAprobados(double notaMinima):** Devuelve la lista de nombres que obtuvieron una nota igual o superior a la mínima.

b) **PromedioGeneral():** Calcula la media aritmética de todas las notas del grupo.

c) **AjustarNotasPorAsistencia():** Añade 5 puntos de bonificación a la nota final de todos los estudiantes con asistencia >= 90% (máximo 100).

d) **BuscarEstudianteConMayorAsistencia():** Devuelve la información del estudiante con el porcentaje de asistencia más alto registrado.

e) **EliminarEstudiantesReprobados(double notaCorte):** Elimina del registro a todos los estudiantes cuya nota sea inferior a la nota de corte y retorna el total de eliminados.

Casos de Prueba Sugeridos: Insertar estudiantes con notas entre 40 y 95 y asistencias entre 70% y 100%.

EQUIPO 7 (3:06 PM – 3:24 PM) — Estructura: *Lista Doblemente Enlazada*

Integrantes: Onel Alejandro Olivera Díaz, Pedro Luis Sarmiento Torres, Daniela González Alemán, María Fernanda Rodríguez Díaz

Problemática: Reproductor de Música 'SoundFlow'

Un reproductor de audio digital necesita controlar la lista de reproducción (playlist). De cada canción se conoce: id, título, artista y duraciónSegundos.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **NavegarAdelanteYAtras():** Permite recorrer y retornar la lista de canciones en orden directo e inverso.

b) **MoverCancionAlInicio(String id):** Ubica la canción especificada al principio de la playlist modificando los enlaces anterior y siguiente.

c) **DuracionTotalPlaylist():** Devuelve el tiempo total de reproducción acumulado en minutos y segundos.

d) **EliminarCancionPorArtista(String artista):** Elimina todas las canciones pertenecientes al artista especificado y retorna la cantidad eliminada.

Casos de Prueba Sugeridos: Insertar 4 canciones. Recorrer en ambos sentidos, mover la última canción al inicio y verificar punteros prev y next.

EQUIPO 8 (3:24 PM – 3:42 PM) — Estructura: *Lista Doblemente Enlazada*

Integrantes: Anyelo Monterrey, Carlos Daniel Hernández Díaz, Laura García Miguel, Alejandro Marrinez Gonzalez

Problemática: Historial de Navegación 'WebBrowser'

Un navegador web requiere administrar el historial de pestañas visitadas para permitir la navegación hacia adelante y hacia atrás. De cada página web se guarda: url, tituloPágina y horaVisita.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **EliminarPaginaPorUrl(String url):** Reenlaza correctamente los nodos anterior y posterior para eliminar una página específica.

b) **MostrarHistorialReciente():** Devuelve la lista de páginas desde la más reciente hasta la más antigua.

c) **BuscarPorPalabraClave(String palabra):** Retorna las URL cuyo título contenga la palabra clave buscada.

d) **VaciarHistorialAntiguo(String horaLimite):** Elimina de la lista las visitas realizadas antes de la hora límite indicada y retorna la cantidad eliminada.

Casos de Prueba Sugeridos: Insertar varias visitas. Eliminar el primer nodo, un nodo intermedio y el último nodo probando la consistencia de punteros.

EQUIPO 9 (3:42 PM – 4:00 PM) — Estructura: *Lista Doblemente Enlazada*

Integrantes: Isnaelys Temerez Rodriguez, Dalís Aladro Acosta, Adrián Felipe Armas García, Carlos Daniel Borrego Chang

Problemática: Sistema de Emergencias 'HospitalEmergency'

La sala de emergencias de un hospital necesita organizar los turnos de atención según la gravedad. De cada paciente se almacena: historialClinico, nombre, nivelPrioridad (1 a 5) y minutosEspera.

Funcionalidades / Métodos a Implementar (1 por estudiante):

a) **AtenderSiguientePaciente():** Elimina y devuelve el primer paciente de la lista.

b) **InsertarPorPrioridad(Paciente nuevo):** Inserta al paciente en la posición adecuada respetando su nivel de prioridad (mayor prioridad al inicio).

c) **PacientesEnEsperaCritica(int minutosLimite):** Devuelve los nombres de los pacientes cuyo tiempo de espera supere el límite.

d) **ReordenarPorTiempoEspera():** Ordena la lista doblemente enlazada de mayor a menor tiempo de espera respetando los enlaces prev y next.

Casos de Prueba Sugeridos: Insertar pacientes con prioridades 2, 5, 1, 4. Verificar orden de atención, tiempos de espera y reordenamiento.

4. Clave de Evaluación General

La evaluación del examen parcial se realizará sobre una escala máxima de **5.0 puntos**, distribuida en los siguientes aspectos:

Criterio	Puntos Máx.	Descripción Detallada
1. Código y Estructura	**2.0 ptos**	Correcta implementación de la estructura asignada en Java, uso adecuado de enlaces/punteros, métodos funcionando correctamente y código limpio.
2. Pruebas y Robustez	**1.0 pto**	Pruebas unitarias/funcionales completas que abarcan flujos positivos, casos negativos y escenarios de borde (lista vacía, elemento no encontrado).
3. Exposición Oral	**1.0 pto**	Dominio técnico del tema, fluidez explicativa, respeto del tiempo asignado (18-20 min) y respuestas precisas a las preguntas planteadas.
4. Repositorio y README	**1.0 pto**	Estructura clara en GitHub, README documentado en lenguaje natural, presentación adjunta y declaración explícita y honesta sobre el uso de IA.

Escala Cualitativa de Calificación:

- **5 Puntos (Excelente):** Cumple de forma sobresaliente con todos los aspectos técnicos, pruebas, documentación, exposición y uso ético/declarado de la IA.
- **4 Puntos (Bien):** Cumple con la mayoría de requerimientos técnicos y exposición, con detalles menores en pruebas o documentación.
- **3 Puntos (Aprobado):** Solución funcional básica pero con fallas leves en casos de borde, respuesta dudosa en preguntas o documentación incompleta.
- **2 Puntos o menos (Suspenso):** El proyecto no compila o no implementa la estructura requerida, faltan métodos clave, incapacidad para responder sobre el código presentado o falta de entrega/declaración del trabajo.